Πανεπιστήμιο Κύπρου
Τμήμα Πληροφορικής

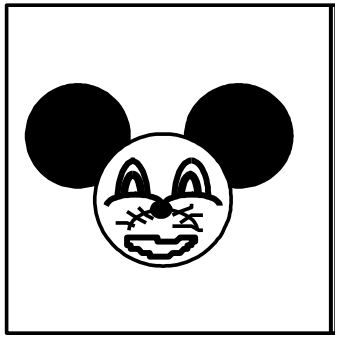
ΕΠΛ445:
Ψηφιακή Επεξεργασία Εικόνας

Εργαστήριο 6

Template Matching

- Είναι μια μέθοδος όπου έχουμε ένα μικρό πρότυπο (template) και ψάχνουμε να βρούμε πού εμφανίζεται μέσα σε μια μεγαλύτερη εικόνα, μέσω ταιριάσματος προτύπου.

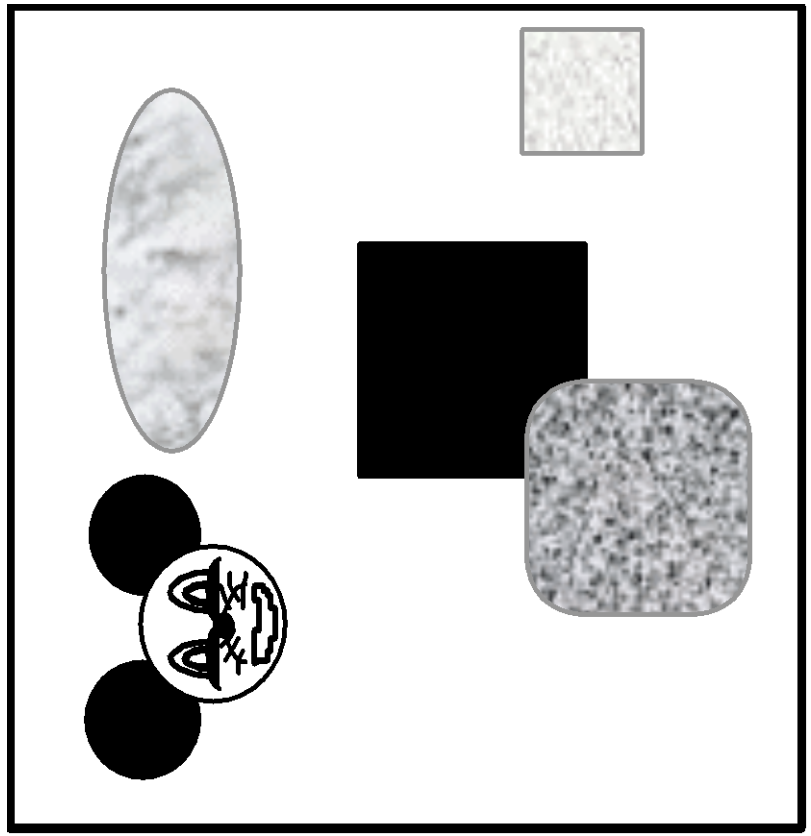
- Περιορισμοί της ταύτισης προτύπου
  - Noise (Θόρυβος)
  - Scaling (Αλλαγή μεγέθους)
  - Stretching (Παραμόρφωση)
  - Rotation (Περιστροφή)



template

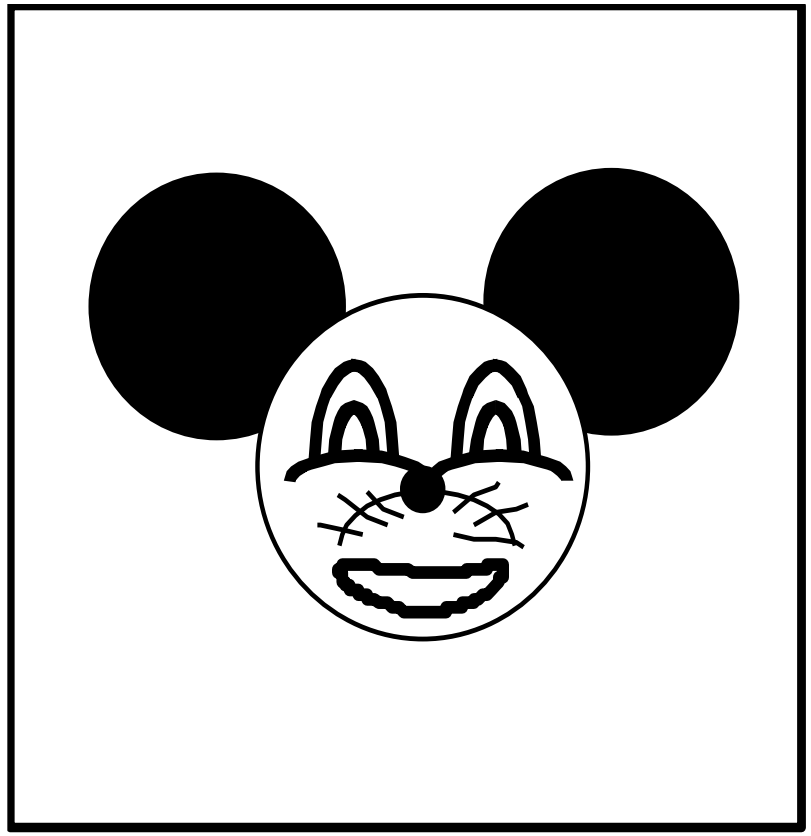
Template Matching

- Το template δεν ταιριάζει γιατί παρουσιάζεται περιστραμμένο.



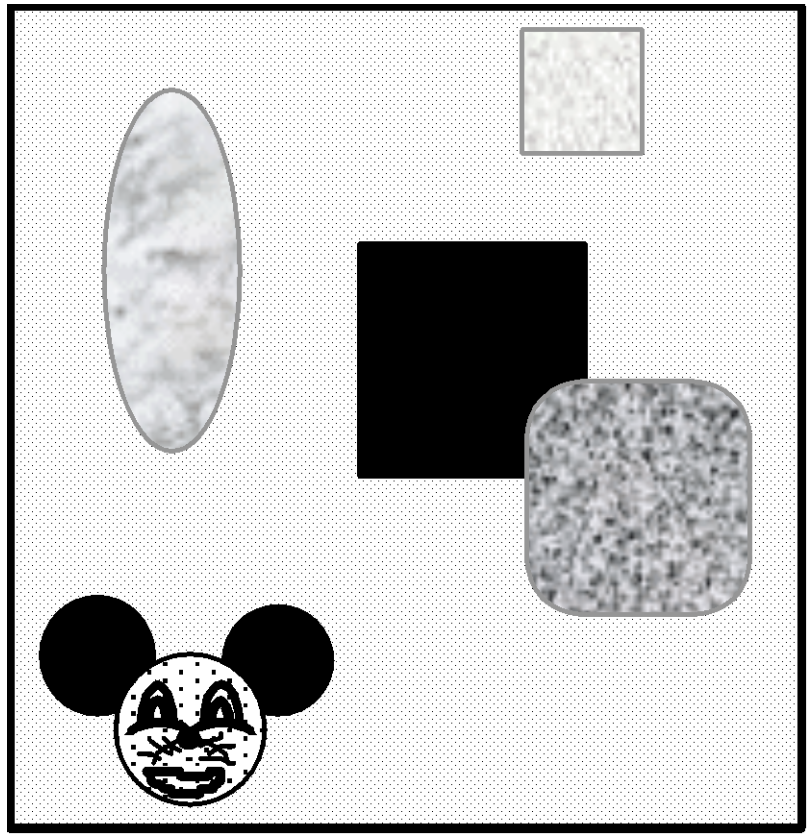
Template Matching

- Το template δεν ταιριάζει γιατί παρουσιάζεται σε μεγέθυνση.



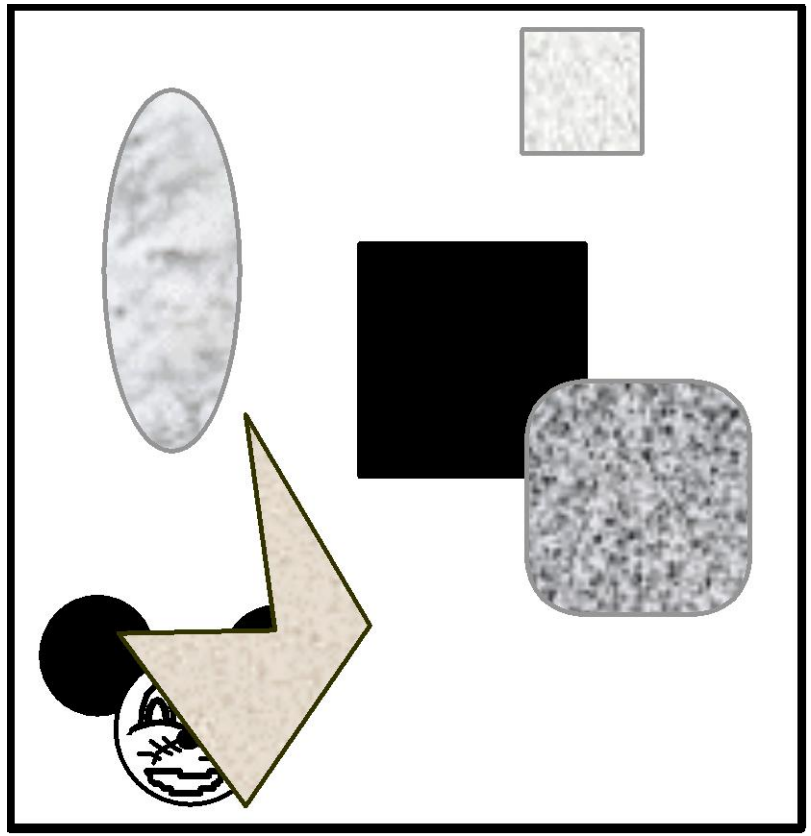
Template Matching

- Το template δεν ταιριάζει γιατί υπάρχει θόρυβος



Template Matching

- Το template δεν ταιριάζει γιατί υπάρχει άλλο αντικείμενο μπροστά του.



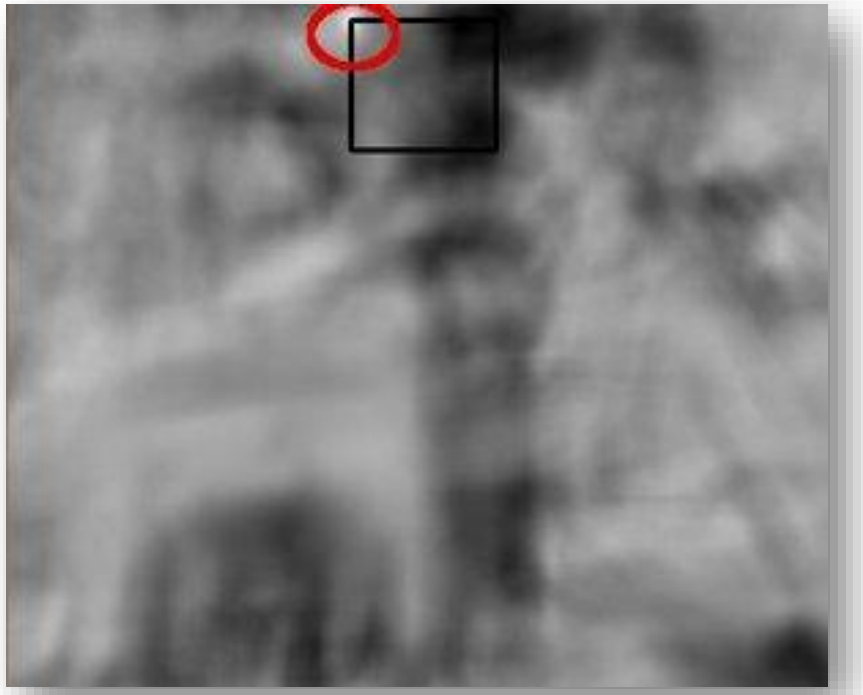
Template Matching

- Πως λειτουργεί ο αλγόριθμος;
  - Έχουμε την source image I και την template image T
  - Για να αναγνωρίσουμε την περιοχή ταιριάσματος, πρέπει να συγκρίνουμε την template image μαζί με την αρχική εικόνα, μετακινώντας την πάνω από αυτή ένα pixel κάθε φορά (από αριστερά προς τα δεξιά και από πάνω προς τα κάτω.
  - Σε κάθε περιοχή, χρησιμοποιώντας μια μετρική, αυτή καθορίζει πόσο «καλό» ή «κακό» είναι το ταίριασμα.

Template Matching

Αλγόριθμος:

- Για κάθε περιοχή της T (template image) πάνω στην I (source image) αποθηκεύουμε το αποτέλεσμα (ένα σκορ ταιριάσματος) της επιλεγμένης μετρικής στον πίνακα R.
- Κάθε σημείο (x,y) στον πίνακα R περιέχει το αποτέλεσμα της μετρικής σε αυτή την περιοχή.
  - Σε αυτό το παράδειγμα ο πίνακας R περιέχει τα αποτελέσματα ταιριάσματος με τη χρήση της μετρικής TM_CCORR_NORMED.
  - Οι πιο φωτεινές περιοχές υποδηλώνουν το «καλύτερο» ταίριασμα. Όπως βλέπετε, το σημείο που καθορίζεται από τον κόκκινο κύκλο είναι αυτό που έχει το καλύτερο ταίριασμα, συνεπώς αυτή η περιοχή που καθορίζεται από το τετράγωνο είναι η περιοχή ταιριάσματος.



R table: result of template matching

R(x,y) = πόσο καλό είναι το match στη θέση (x,y)

Template Matching in OpenCV

- Εφαρμογή template matching:
  cv2.matchTemplate()
- Εύρεση της μεγαλύτερης (ή της μικρότερης – εξαρτάται από τη μετρική που χρησιμοποιούμε) τιμής στον πίνακα R:
  cv2.minMaxLoc()
  - Η συνάρτηση βρίσκει τις περιοχές των μεγαλύτερων ή μικρότερων τιμών στον πίνακα R (σε συντεταγμένες x,y)
  - Η μέθοδος δεν λειτουργεί σε πίνακες που έχουν πέραν του ενός καναλιού (π.χ. BGR)
    - Ένας πολυκαναλικός πίνακας θα πρέπει να σπάσει σε πίνακες ενός καναλιού [π.χ. με τη χρήση της μεθόδου split()]

Αλγόριθμοι Template Matching στο OpenCV

a. **method=CV_TM_SQDIFF**

$$R(x,y) = \sum_{x',y'} (T(x',y') - I(x+x',y+y'))^2$$

b. **method=CV_TM_SQDIFF_NORMED**

$$R(x,y) = \frac{\sum_{x',y'} (T(x',y') - I(x+x',y+y'))^2}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x',y+y')^2}}$$

c. **method=CV_TM_CCORR**

$$R(x,y) = \sum_{x',y'} (T(x',y') \cdot I(x+x',y+y'))$$

d. **method=CV_TM_CCORR_NORMED**

$$R(x,y) = \frac{\sum_{x',y'} (T(x',y') \cdot I(x+x',y+y'))}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x',y+y')^2}}$$

e. **method=CV_TM_CCOEFF**

$$R(x,y) = \sum_{x',y'} (T'(x',y') \cdot I(x+x',y+y'))$$

where

$$T'(x',y') = T(x',y') - 1/(w \cdot h) \cdot \sum_{x'',y''} T(x'',y'')$$
$$I'(x+x',y+y') = I(x+x',y+y') - 1/(w \cdot h) \cdot \sum_{x'',y''} I(x+x'',y+y'')$$

f. **method=CV_TM_CCOEFF_NORMED**

$$R(x,y) = \frac{\sum_{x',y'} (T'(x',y') \cdot I'(x+x',y+y'))}{\sqrt{\sum_{x',y'} T'(x',y')^2 \cdot \sum_{x',y'} I'(x+x',y+y')^2}}$$

Template Matching στο OpenCV

```
methods = ['cv2.TM_CCOEFF', 'cv2.TM_CCOEFF_NORMED', 'cv2.TM_CCORR',
           'cv2.TM_CCORR_NORMED', 'cv2.TM_SQDIFF', 'cv2.TM_SQDIFF_NORMED']

for meth in methods:
           img = img2.copy()
           # Enumerate the template matching operation
           method = eval(meth)
            # Apply template matching
           res = cv2.matchTemplate(img,template,method)
           min_val, max_val, min_loc, max_loc = cv2.minMaxLoc(res)

           # If the method is TM_SQDIFF or TM_SQDIFF_NORMED, take minimum
           if method in [cv2.TM_SQDIFF, cv2.TM_SQDIFF_NORMED]:
                      top_left = min_loc
           else:
                      top_left = max_loc
           bottom_right = (top_left[0] + w, top_left[1] + h)

           cv2.rectangle(img,top_left, bottom_right, 255, 2)
```

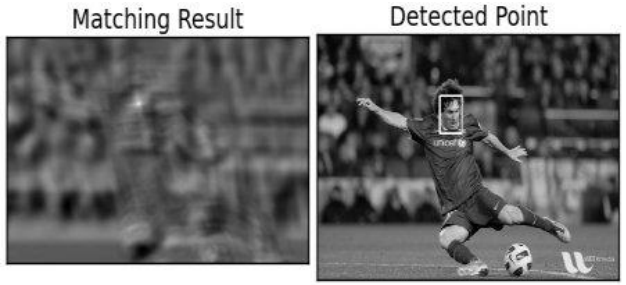
w, h = template.shape

Img – source image
Top_left – πάνω αριστερό σημείο του τετραγώνου
Bottom_right – κάτω δεξί σημείο του τετραγώνου
255 – χρώμα του τετραγώνου (εδώ λευκό)
2 – πάχος γραμμής του τετραγώνου

More information click here

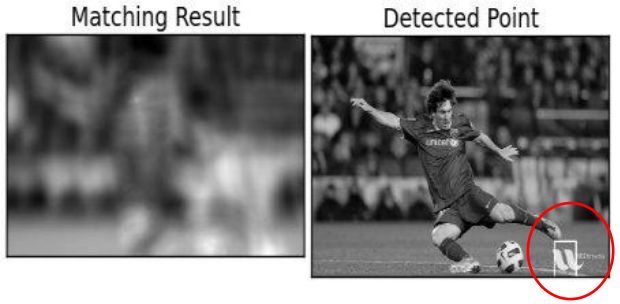
Template Matching στο OpenCV

- cv2.TM_CCOEFF



- cv2.TM_CCOEFF_NORMED

- cv2.TM_CCORR (not a good method)



- cv2.TM_CCORR_NORMED

- cv2.TM_SQDIFF

- cv2.TM_SQDIFF_NORMED

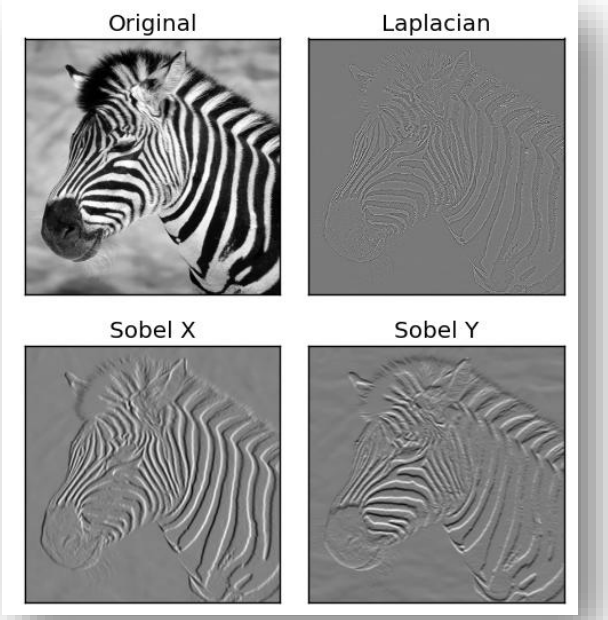
Image Gradients (από προηγούμενο εργαστήριο)

```
import cv2
from matplotlib import pyplot as plt

img = cv2.imread('zebra.jpg',0)

# Apply image gradients or high-pass filters
laplacian = cv2.Laplacian(img,cv2.CV_64F)
sobelx = cv2.Sobel(img,cv2.CV_64F,1,0,ksize=5)
sobely = cv2.Sobel(img,cv2.CV_64F,0,1,ksize=5)

plt.subplot(2,2,1),plt.imshow(img,cmap = 'gray')
plt.title('Original'), plt.xticks([]), plt.yticks([])
plt.subplot(2,2,2),plt.imshow(laplacian,cmap = 'gray')
plt.title('Laplacian'), plt.xticks([]), plt.yticks([])
plt.subplot(2,2,3),plt.imshow(sobelx,cmap = 'gray')
plt.title('Sobel X'), plt.xticks([]), plt.yticks([])
plt.subplot(2,2,4),plt.imshow(sobely,cmap = 'gray')
plt.title('Sobel Y'), plt.xticks([]), plt.yticks([])
plt.show()
```

Ανίχνευση ασυνέχειας στη φωτεινότητα της εικόνας με τη βοήθεια υπολογισμού της κλίσης (gradient) της εικόνας. Η κλίση της εικόνας είναι το πλάτος του διανύσματος που παρέχει πληροφορία για τα τις μεταβολές της φωτεινότητας.

Το φίλτρο Sobel είναι φίλτρο Gaussian smoothing με μια διαφοροποίηση, έτσι ώστε να είναι πιο ανθεκτικό στον θόρυβο. Μπορούμε να καθορίσουμε την κατεύθυνση των παραγώγων που πρέπει να ληφθούν, κάθετη ή οριζόντια (Yorder και Xorder αντίστοιχα). Μπορούμε επίσης να καθορίσουμε το μέγεθος του πυρήνα.

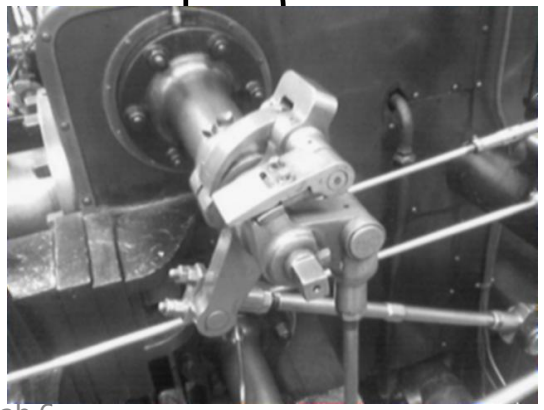
More info click here

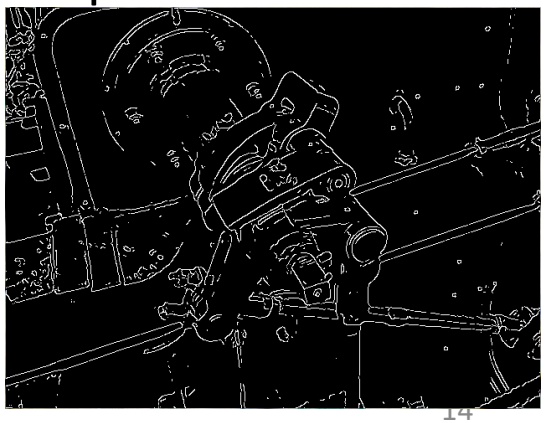


Αλγόριθμος Canny στο OpenCV

Βήματα αλγόριθμου

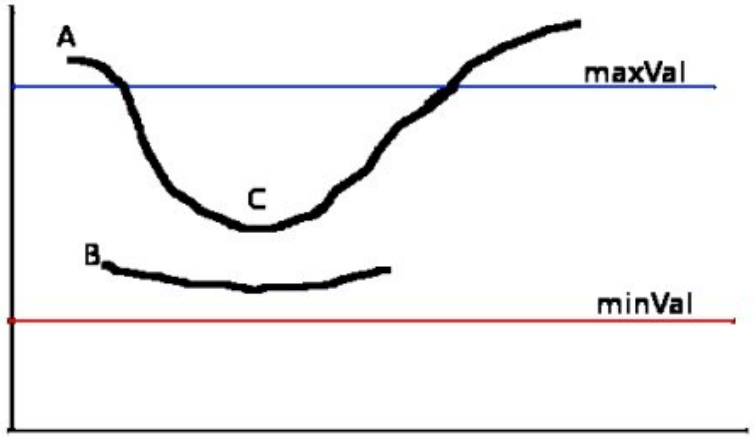
1. Εξομάλυνση εικόνας (Image smoothing): Αφαίρεση θορύβου με χρήση Gaussian φίλτρου.
2. Υπολογισμός κλίσης (gradient): Υπολογίζεται το μέτρο και η κατεύθυνση της κλίσης (1η παράγωγος) χρησιμοποιώντας τον τελεστή Sobel και στις δύο κατευθύνσεις.
3. Κάθε pixel ελέγχεται αν είναι τοπικό μέγιστο στη γειτονιά του προς την κατεύθυνση της κλίσης. Αν είναι, προχωρά στο επόμενο στάδιο. Αν δεν είναι, τίθεται στο μηδέν (καταστολή).
4. Κατωφλίωση (Thresholding): Εφαρμόζεται κατώφλι για τον εντοπισμό των τελικών ακμών.



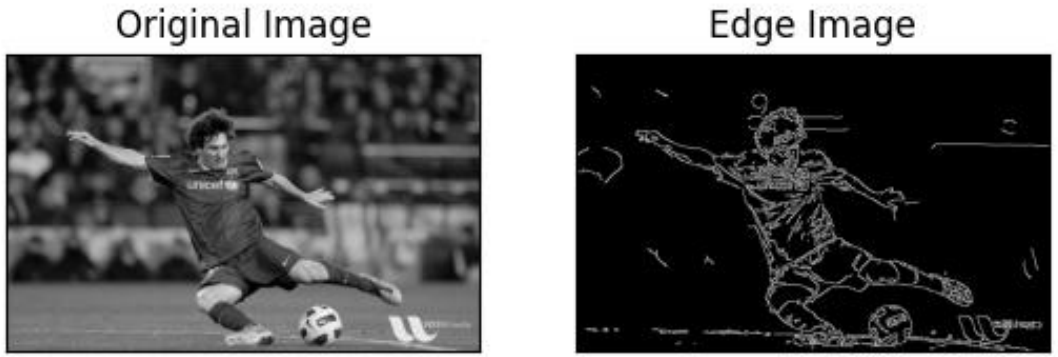


Ανίχνευση Ακμών - Αλγόριθμος Canny

- Αλγόριθμος Canny
  - Χρειαζόμαστε δύο τιμές κατωφλίου, minVal και maxval. Οποιαδήποτε pixels με κλίση πάνω από maxval είναι σίγουρο ότι θα είναι ακμές και αυτά με κλίση κάτω από minVal είναι βέβαιο ότι θα είναι μη-ακμές, έτσι απορρίπτονται.
  - Εκείνα που βρίσκονται μεταξύ αυτών των δύο ορίων ταξινομούνται ως ακμές ή μη-ακμές ανάλογα με τη σύνδεσή τους. Αν είναι συνδεδεμένα με τα pixels που ανήκουν σε ακμές, θεωρούνται ότι είναι ακμές. Διαφορετικά απορρίπτονται, όπως φαίνεται στην διπλανή εικόνα.
  - Το pixel A έχει κλίση πάνω από maxval, οπότε θεωρείται ως βέβαιη ακμή. Αν και το pixel C έχει κλίση κάτω από maxval, είναι συνδεδεμένο στο pixel A, έτσι θεωρείται έγκυρη ακμή και έτσι παίρνουμε τη πλήρη καμπύλη.
  - Το pixel B, αν και έχει κλίση πάνω από minVal και είναι στην είναι ίδια περιοχή με εκείνη του pixel C, δεν είναι συνδεδεμένο με κάποιο άλλο που είναι ακμή, έτσι απορρίπτεται.



Αλγόριθμος Canny στο OpenCV

```
import cv2
from matplotlib import pyplot as plt

img = cv2.imread('messi.jpg',0)
edges = cv2.Canny(img,100,200)

plt.subplot(121),plt.imshow(img,cmap = 'gray')
plt.title('Original Image'), plt.axis('off')
plt.subplot(122),plt.imshow(edges,cmap = 'gray')
plt.title('Edge Image'), plt.axis('off')

plt.show()
```

Βάζουμε όρια για ένταση φωτεινότητας και ο αλγόριθμος τα μετατρέπει σε όρια κλίσης.